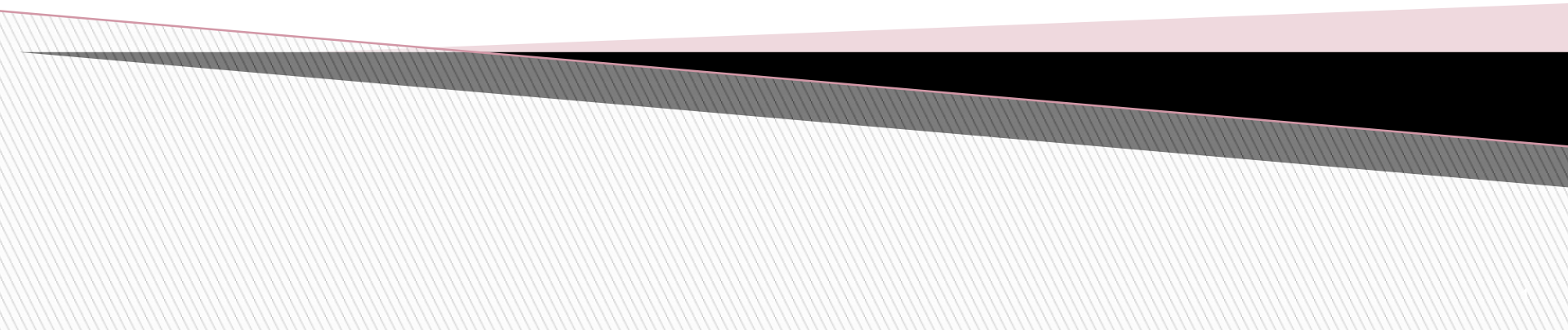


EE-2003 Computer Organization and Assembly Language

Fall 2026

Department of Computer Science/Software Engineering/Artificial
Intelligence

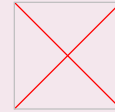
The bottom of the slide features a decorative graphic consisting of several overlapping, wavy, horizontal bands. From top to bottom, the bands are: a light pink band, a solid black band, a dark grey band with fine diagonal hatching, and a light grey band with fine diagonal hatching. The lines between the bands are slightly irregular, giving it a hand-drawn or layered appearance.

Why This Course Matters



Computer Science

Your algorithms only run fast if you know what the machine underneath is doing.



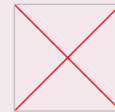
Computer Engineering

Buses, registers and memory maps are the hardware you will design and debug.



Software Engineering

Performance and memory bugs live below the language you write in.



Artificial Intelligence

AI speed is a memory-bandwidth problem: caches, precision and data movement.

Compilers, operating systems and ML frameworks are all built on this layer.

Course Information

- Text and Reference Books
 - Course Outline available on Google Class Room
- Course Objectives and outcomes
 - Available on Google Class Room
- Lecture material will be available on GCR
- Regularly review the lectures discussed in the class
- Prior knowledge of Courses required
 - ITC and DLD

From 4-bit Intel 4004 to Modern Multi-Core x86-64 CPUs

Year	Processor	Key Features
1971	Intel 4004	First commercial microprocessor; 4-bit; 640 bytes addressable memory; used in calculators.
1972	Intel 8008	First 8-bit processor; addressed up to 16 KB memory; used in early hobbyist computers.
1974	Intel 8080	Popular 8-bit processor; addressed 64 KB memory; used in Altair 8800 (kickstarted personal computing).
1976	Intel 8085	Enhanced 8-bit processor; simpler single +5V power supply; instruction set compatible with 8080.
1978	Intel 8086	First 16-bit processor; 1 MB memory; foundation of x86 architecture.
1979	Intel 8088	16-bit processor with 8-bit external data bus (cheaper design); used in IBM PC (1981).
1982	Intel 80286 (286)	16-bit; introduced protected mode; could address 16 MB memory; used in IBM PC/AT.
1985	Intel 80386 (386)	First 32-bit x86 processor (IA-32); supported virtual memory and multitasking; 4 GB address space.
1989	Intel 80486 (486)	Integrated floating-point unit (FPU); on-chip cache; higher performance.
1993	Intel Pentium	Superscalar architecture (multiple instructions per clock cycle); strong floating-point performance.
2003	AMD64 / x86-64	64-bit extension to x86; backward compatible with 32-bit; larger memory addressing.
Present	Intel Core / AMD Ryzen	Multi-core, hyper-threading, integrated GPUs, advanced power management, high-performance computing.

Evolution of x86 Architecture

Year	Processor	Key Features
1978	Intel 8086	First 16-bit processor, foundation of x86 family
1982	Intel 80286	Introduced protected mode, better memory management
1985	Intel 80386 (i386)	32-bit architecture (IA-32), virtual memory support
1989	Intel 80486 (i486)	Integrated FPU, improved performance
1993	Intel Pentium	Superscalar architecture (multiple instructions per cycle)
2003	AMD64 / x86-64	64-bit extension of x86, larger memory addressing
Present	Intel Core / AMD Ryzen	Modern multi-core x86-64 processors

Feature	16-bit Processor	32-bit Processor	64-bit Processor
Register Size	16 bits (2 bytes)	32 bits (4 bytes)	64 bits (8 bytes)
Maximum Integer Value	$2^{16} - 1 = 65,535$	$2^{32} - 1 \approx 4.29 \times 10^9$	$2^{64} - 1 \approx 1.84 \times 10^{19}$
Addressable Memory	$2^{16} = 64$ KB (8086 used segmentation for 1 MB)	$2^{32} = 4$ GB	$2^{64} \approx 16$ EB (practically a few TB supported)
Instruction Set Width	Operates on 16-bit data	Operates on 32-bit data	Operates on 64-bit data
Examples	Intel 8086, 80286	Intel 80386, Pentium (32-bit mode)	Intel Core i7, AMD Ryzen (x86-64)

Table 1: Comparison of 16-bit, 32-bit, and 64-bit Processors

Languages Generations!

Levels of Programming Languages

Level	Description	Examples
Low-Level Languages	Close to machine hardware; require detailed knowledge of computer architecture; fast execution but hard to program and maintain.	Machine Language (binary code), Assembly Language (x86, ARM, 8086 assembly)
Middle-Level Languages	Bridge between low-level and high-level; provide access to hardware while supporting abstraction; used for system programming.	C, C++
High-Level Languages	Abstracted from hardware; focus on ease of programming, readability, and portability; automatically manage low-level details.	Python, Java, C#, JavaScript, Ruby

Some Basics- Quick Review

Lecture Overview

- Numbering & Coding Systems
- Digital Primer
- Semiconductor Memory

Numbering & Coding Systems

- Decimal, Binary and Hexadecimal Number Systems
- Interconversion Between these Systems
 - converting from decimal to binary and binary to decimal
 - converting between binary and hex
 - converting from hex to decimal
 - addition/subtraction of binary and hex numbers

Express the decimal number -39 as an 8-bit number in the sign-magnitude, 1's complement, and 2's complement forms.

Solution

First, write the 8-bit number for $+39$.

00100111

In the *sign-magnitude form*, -39 is produced by changing the sign bit to a 1 and leaving the magnitude bits as they are. The number is

10100111

In the *1's complement form*, -39 is produced by taking the 1's complement of $+39$ (00100111).

11011000

In the *2's complement form*, -39 is produced by taking the 2's complement of $+39$ (00100111) as follows:

11011000	1's complement
+ 1	
11011001	2's complement

Related Problem

Determine the decimal values of the signed binary numbers expressed in 2's complement:

- (a) 01010110 (b) 10101010

Solution

- (a) The bits and their powers-of-two weights for the positive number are as follows:

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	0	1	1	0

Summing the weights where there are 1s,

$$64 + 16 + 4 + 2 = +86$$

- (b) The bits and their powers-of-two weights for the negative number are as follows. Notice that the negative sign bit has a weight of $-2^7 = -128$.

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	0	1	0	1	0

Summing the weights where there are 1s,

$$-128 + 32 + 8 + 2 = -86$$

Convert the following binary numbers to hexadecimal:

- (a) 1100101001010111 (b) 111111000101101001

Solution

$$(a) \quad \underbrace{1100}_{\downarrow C} \underbrace{1010}_{\downarrow A} \underbrace{0010}_{\downarrow 5} \underbrace{1011}_{\downarrow 7} = \mathbf{CA57}_{16}$$

$$(b) \quad \underbrace{0011}_{\downarrow 3} \underbrace{1111}_{\downarrow F} \underbrace{1000}_{\downarrow 1} \underbrace{1011}_{\downarrow 6} \underbrace{01001}_{\downarrow 9} = \mathbf{3F169}_{16}$$

Two zeros have been added in part (b) to complete a 4-bit group at the left.

Determine the binary numbers for the following hexadecimal numbers:

(a) $10A4_{16}$ (b) $CF8E_{16}$ (c) 9742_{16}

Solution

(a) $\begin{array}{cccc} 1 & 0 & A & 4 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 10000 & 10100 & 100 & \end{array}$

(b) $\begin{array}{cccc} C & F & 8 & E \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1100 & 1111 & 1000 & 1110 \end{array}$

(c) $\begin{array}{cccc} 9 & 7 & 4 & 2 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1001 & 0111 & 0100 & 0010 \end{array}$

In part (a), the MSB is understood to have three zeros preceding it, thus forming a 4-bit group.

Convert the following hexadecimal numbers to decimal:

- (a) $E5_{16}$ (b) $B2F8_{16}$

Solution

Recall from Table 2–3 that letters A through F represent decimal numbers 10 through 15, respectively.

$$(a) \ E5_{16} = (E \times 16) + (5 \times 1) = (14 \times 16) + (5 \times 1) = 224 + 5 = 229_{10}$$

$$\begin{aligned}(b) \ B2F8_{16} &= (B \times 4096) + (2 \times 256) + (F \times 16) + (8 \times 1) \\ &= (11 \times 4096) + (2 \times 256) + (15 \times 16) + (8 \times 1) \\ &= 45,056 + 512 + 240 + 8 = 45,816_{10}\end{aligned}$$

$$\begin{array}{r} 58_{16} \\ + 22_{16} \\ \hline 7A_{16} \end{array}$$

right column: $8_{16} + 2_{16} = 8_{10} + 2_{10} = 10_{10} = A_{16}$

left column: $5_{16} + 2_{16} = 5_{10} + 2_{10} = 7_{10} = 7_{16}$

$$\begin{array}{r} 2B_{16} \\ + 84_{16} \\ \hline AF_{16} \end{array}$$

right column: $B_{16} + 4_{16} = 11_{10} + 4_{10} = 15_{10} = F_{16}$

left column: $2_{16} + 8_{16} = 2_{10} + 8_{10} = 10_{10} = A_{16}$

$$\begin{array}{r} DF_{16} \\ + AC_{16} \\ \hline 18B_{16} \end{array}$$

right column: $F_{16} + C_{16} = 15_{10} + 12_{10} = 27_{10}$

$27_{10} - 16_{10} = 11_{10} = B_{16}$ with a 1 carry

left column: $D_{16} + A_{16} + 1_{16} = 13_{10} + 10_{10} + 1_{10} = 24_{10}$

$24_{10} - 16_{10} = 8_{10} = 8_{16}$ with a 1 carry

...

Digital Primer

- What is digital logic?
- Logic gates are put together to form simple digital circuits, i.e Adders, subtractor, decoder etc

SemiConductor Memory

Bit				0
Nibble				0000
Byte		0000	0000	
Word	0000	0000	0000	0000

A *kilobyte* is 2^{10} bytes, which is 1024 bytes. The abbreviation K is often used to represent kilobytes. A *megabyte*, or *meg* as some call it, is 2^{20} bytes. That is a little over 1 million bytes; it is exactly 1,048,576 bytes. Moving rapidly up the scale in size, a *gigabyte* is 2^{30} bytes (over 1 billion), and a *terabyte* is 2^{40} bytes (over 1 trillion). As an example of how some of these terms are used, suppose that a given computer has 16 megabytes of memory. That would be 16×2^{20} , or $2^4 \times 2^{20}$, which is 2^{24} . Therefore 16 megabytes is 2^{24} bytes.

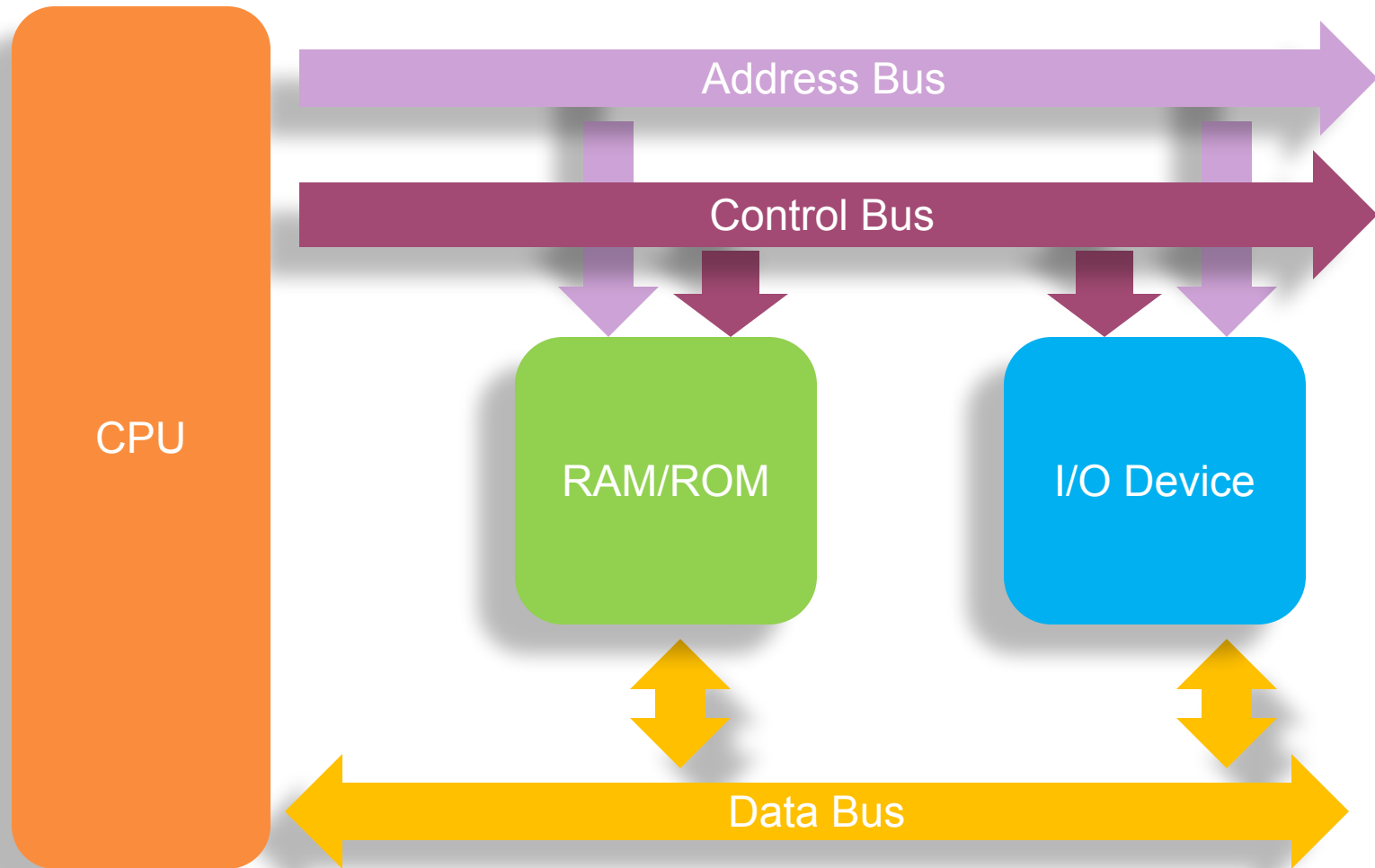
Important Terminologies !

Multiples of Bytes		
Unit (Symbol)	Value (SI)	Value (Binary)
Kilobyte (kB)	10^3	2^{10}
Megabyte (MB)	10^6	2^{20}
Gigabyte (GB)	10^9	2^{30}
Terabyte (TB)	10^{12}	2^{40}
Petabyte (PB)	10^{15}	2^{50}
Exabyte (EB)	10^{18}	2^{60}
Zettabyte (ZB)	10^{21}	2^{70}
Yottabyte (YB)	10^{24}	2^{80}

RAM & ROM

Two types of memory commonly used in microcomputers are *RAM*, which stands for “random access memory” (sometimes called *read/write memory*), and *ROM*, which stands for “read-only memory.” RAM is used by the computer for temporary storage of programs that it is running. That data is lost when the computer is turned off. For this reason, RAM is sometimes called *volatile memory*. ROM contains programs and information essential to operation of the computer. The information in ROM is permanent, cannot be changed by the user, and is not lost when the power is turned off. Therefore, it is called *nonvolatile memory*.

Organization of Computers



Organization of Computers

- Address Bus
 - Size of Memory accessible by Processor
 - 64 K locations = 16 bit address
 - 4 GB = 32 bit; 16 EB = 64 bit
- Data Bus
 - Chunk of Data accessible
 - 8-bit data Bus accesses 1 byte
 - 16-bit data Bus accesses 2 bytes
 - 64-bit data Bus accesses 8 bytes
- RAM
 - Stores program and data which is not permanent
- ROM
 - Stores permanent code and data
 - Now flash / UEFI firmware

Table 0-4: Powers of 2

x	2^x
10	1K
11	2K
12	4K
13	8K
14	16K
15	32K
16	64K
17	128K
18	256K
19	512K
20	1M
21	2M
22	4M
23	8M
24	16M
25	32M
26	64M
27	128M

Internal Organization of Computers

- ❑ CPU – Process information stored in memory
- ❑ I/O – Provides mean of communicating with CPU
- ❑ Memory- RAM & ROM
 - RAM is a volatile and is used by the computer for temporary storage of a program that is running.
 - ROM is a non-volatile, information is permanent and can not be changed by the user.
- ❑ CPU is connected to I/O's & Memory through buses.

Address, Data & Control Bus

- ❑ Address Bus – For a device to be recognized by the CPU it must be assigned an address.

Address assigned to a given device must be unique.

CPU puts the address on the address bus, and the decoding circuitry finds the device.

- ❑ Data Bus- CPU uses data bus either to get data from or to send the device.
- ❑ Control Bus –CPU use the control bus to provide the read or write signal to the device to indicate if the CPU is asking for information or sending information.

More about Data Bus..

- ▣ More data buses available the better and expensive the CPU will be..
average size of data buses varies between 8 & 64
- ❑ Data buses are bidirectional.
- ❑ Processing power of CPU is related to the size of its buses.
- ❑ Modern CPUs: 64-bit external data bus; 128/256/512-bit SIMD paths (SSE/AVX).
- ❑ A DDR5 channel is 64 bits wide, split into two 32-bit sub-channels.

More about Address Bus

- More address buses present, the larger device can be addressed. Number of address buses determines the number of locations with which it can communicate

$$\text{number of location} = 2^x$$

x are number of address lines

- ❑ CPU with 16 address lines can provide 2^{16} of addressable memory.
- ❑ x86-64 uses 48 address lines for virtual and up to 52 for physical memory.
- ❑ 48 lines = 256 TB of address space, far beyond installed RAM.

Address Bus Width: 16 to 64 Bits

Address Lines	Addressable Locations	Where You See It
16	$2^{16} = 64 \text{ K}$	Intel 8085 (1976)
20	$2^{20} = 1 \text{ MB}$	Intel 8086 (1978)
32	$2^{32} = 4 \text{ GB}$	IA-32 / Pentium
48	$2^{48} = 256 \text{ TB}$	x86-64 virtual address
52	$2^{52} = 4 \text{ PB}$	x86-64 physical limit
64	$2^{64} = 16 \text{ EB}$	Architectural ceiling

Doubling the address bus does not double memory — it squares the reach.

CPU and Its relation to RAM & ROM

- ❑ For a CPU to process information the data must be present in ROM or RAM.

ROM provides fixed and permanent information such as tables for character patterns or initialization programs.

- ❑ Today that ROM is flash memory holding UEFI firmware: non-volatile, but field-updatable.
- ❑ RAM stores temporary information that can be changed with time such as various versions of OS and application packages etc.
- ❑ CPU gets information to be processed from RAM or ROM, if its not there it request the secondary storage such hard drives and transfer the information to RAM.

The Modern Memory Hierarchy

Level	Typical Size	Access Time	Managed By
L1 cache	32–64 KB / core	~1 ns	Hardware
L2 cache	0.5–2 MB / core	~4 ns	Hardware
L3 cache	8–64 MB shared	~15 ns	Hardware
Main memory (DDR5)	8–64 GB	~80 ns	Operating system
NVMe SSD	256 GB – 4 TB	~50 μ s	Operating system

Memory Capacity

- ❏ Number of bits that semiconductor memory can store is chip capacity.

memory can be in units of kbits, Mbits ..

Memory Organization

- ❏ A memory chip contains 2^x locations -
 x are address pins
- ❏ Each location contains y bits -
 y is the number of data pins on the chip
- ❏ Entire chip will contain $2^x * y$ bits

Examples.. Finding Capacity & Organization of memory..

Example 0-12

A given memory chip has 12 address pins and 4 data pins. Find:

(a) the organization, and (b) the capacity.

Solution:

- (a) This memory chip has 4,096 locations ($2^{12} = 4,096$), and each location can hold 4 bits of data. This gives an organization of $4,096 \times 4$, often represented as $4K \times 4$.
- (b) The capacity is equal to 16K bits since there is a total of 4K locations and each location can hold 4 bits of data.

Example 0-13

A 512K memory chip has 8 pins for data. Find:

(a) the organization, and (b) the number of address pins for this memory chip.

Solution:

- (a) A memory chip with 8 data pins means that each location within the chip can hold 8 bits of data. To find the number of locations within this memory chip, divide the capacity by the number of data pins. $512K/8 = 64K$; therefore, the organization for this memory chip is $64K \times 8$.
- (b) The chip has 16 address lines since $2^{16} = 64K$.

Capacity & Org. of Some UV-EPROM Chips...

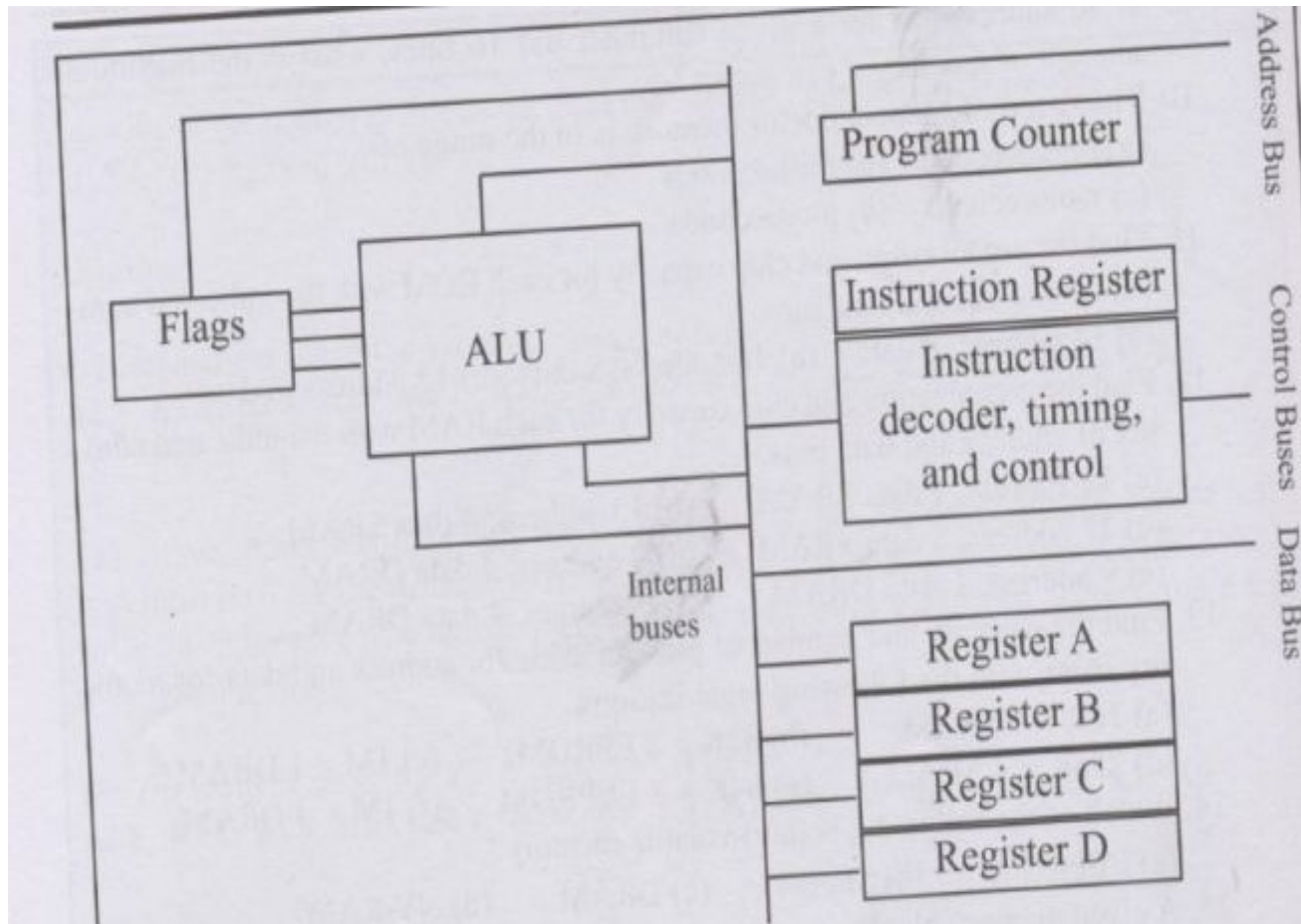
Table 0-5: Some UV-EPROM Chips

Part #	Capacity	Org.	Access	Pins	V _{pp}
2716	16K	2K × 8	450 ns	24	25 V
2732	32K	4K × 8	450 ns	24	25 V
2732A-20	32K	4K × 8	200 ns	24	21 V
27C32-1	32K	4K × 8	450 ns	24	12.5 V CMOS
2764-20	64K	8K × 8	200 ns	28	21 V
2764A-20	64K	8K × 8	200 ns	28	12.5 V
27C64-12	64K	8K × 8	120 ns	28	12.5 V CMOS
27128-25	128K	16K × 8	250 ns	28	21 V
27C128-12	128K	16K × 8	120 ns	28	12.5 V CMOS
27256-25	256K	32K × 8	250 ns	28	12.5 V
27C256-15	256K	32K × 8	150 ns	28	12.5 V CMOS
27512-25	512K	64K × 8	250 ns	28	12.5 V
27C512-15	512K	64K × 8	150 ns	28	12.5 V CMOS
27C010-15	1024K	128K × 8	150 ns	32	12.5 V CMOS
27C020-15	2048K	256K × 8	150 ns	32	12.5 V CMOS
27C040-15	4096K	512K × 8	150 ns	32	12.5 V CMOS

Inside CPU

- ❑ The CPU uses **registers** to store information temporarily
- ❑ Registers can be 8-bit, 16-bit, 32-bit or even 64-bit
- ❑ More and bigger registers, the better the CPU.
- ❑ CPU cost is increased with more and bigger registers.
- ❑ CPU is having **ALU** for performing arithmetic functions
- ❑ **Program counter** in the CPU points to the address of the next instruction to be executed.
- ❑ **Instruction decoder** in CPU interpret the instruction fetch into the CPU
- ❑ CPU capable of understanding more instructions requires more transistors to design

Example Internal Working of CPU



Adding three numbers

- ❑ Add numbers 21, 42 and 12
- ❑ Which system unit will add the two numbers?
(Register, ALU, Control Unit)

The ALU

- ❑ Before the ALU can add the numbers, where should the numbers be temporarily stored?

Registers

- ❑ Assume that CPU has four registers A, B, C & D

Adding three numbers

- ❑ What is the range of memory address and size of data if the CPU can access data through an 8-bit data bus and 16-bit address bus?

It can access one byte of instruction or data

Address range is 0000 to FFFF H

Adding three numbers

- ❑ The CPU performs the following actions to add the three numbers
 - Put 21 into A register
 - Add to register A the value 42
 - Add to register A the value 12
- ❑ The program instructions which implement the above actions are stored in the memory
- ❑ The code to move the value to register A is
 - 1011 0000 (B0 H)
- ❑ The code to add the value to register A is
 - 0000 0100 (04 H)

Adding three numbers

Action	Code	Data
Move value 21 H into reg. A	B0 H	21 H
Add value 42 H to reg. A	04 H	42 H
Add value 12 H to reg. A	04 H	12 H

- ❑ Where should the program be stored?

In the Memory

- ❑ At which address should the program be stored?

The first address of the memory

But the start address can be any address

- ❑ Assume first address where instruction is stored is 1400 H

Adding three numbers

- ❑ What is the size of the program? (in bytes)

Seven bytes

AddressInstruction

1400 B0

1401 21

1402 04

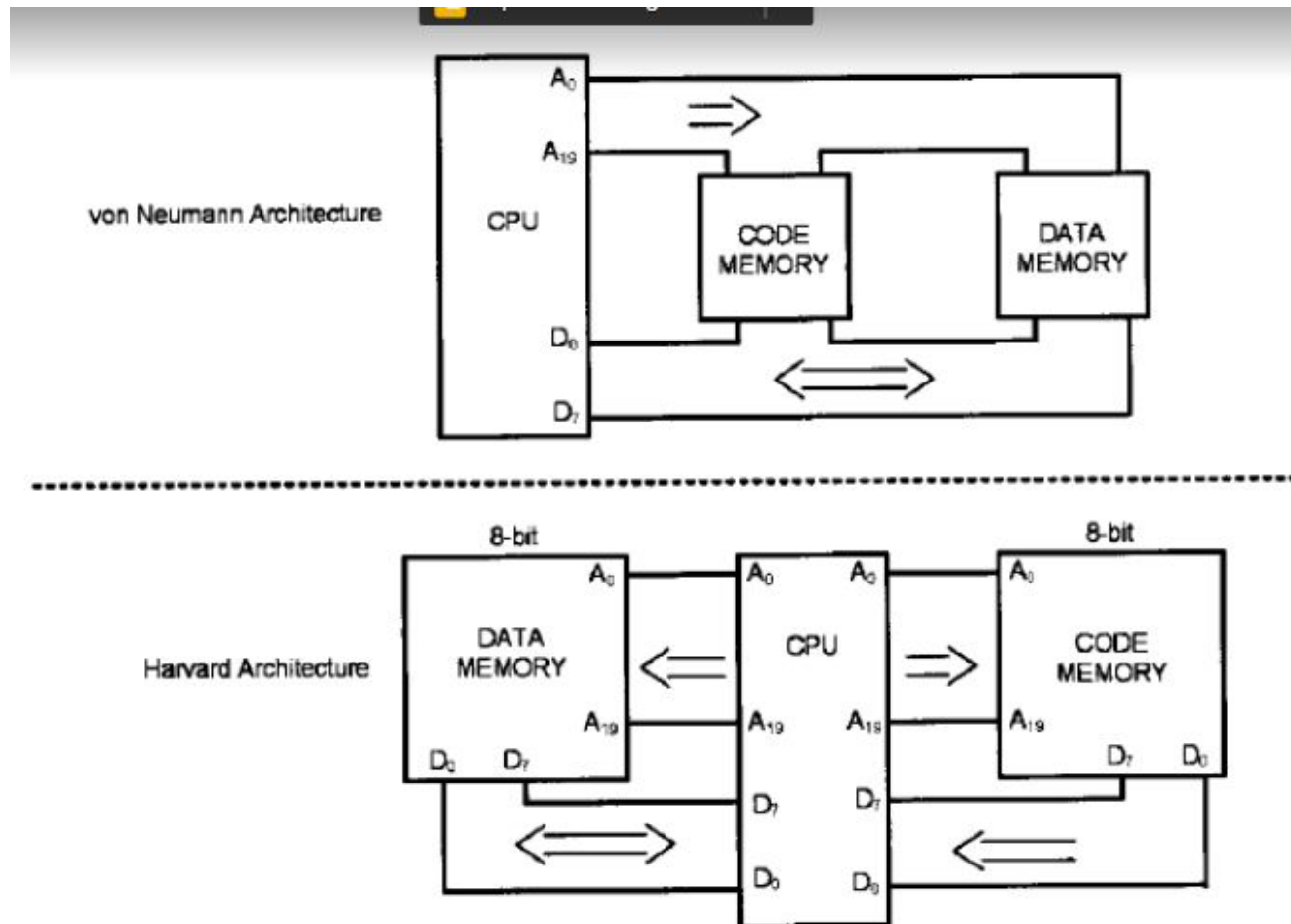
1403 42

1404 04

1405 12

1406 F4

Von Neumann vs. Harvard Architecture



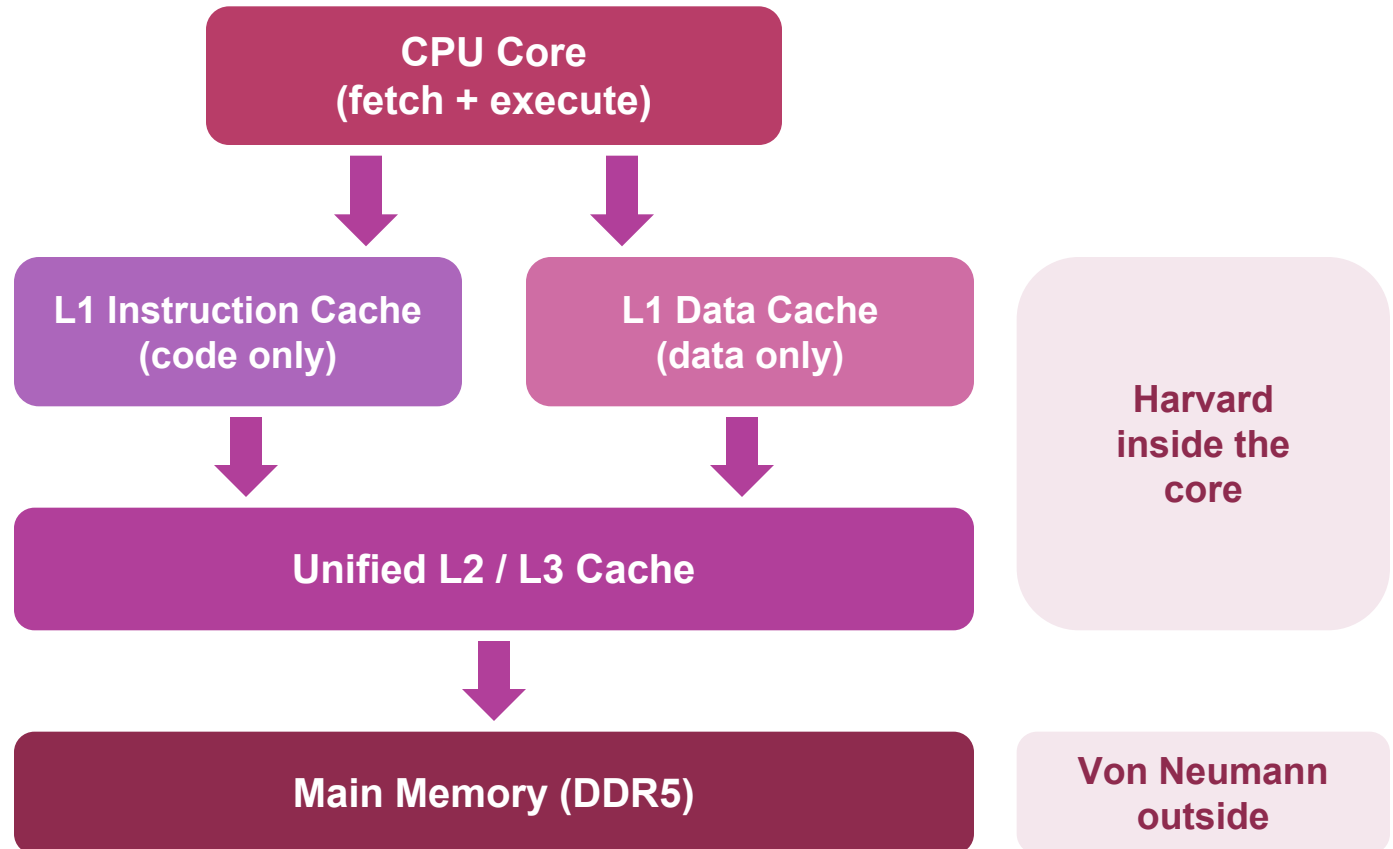


- ❑ Every microprocessor must have memory space to store program (code) and data.
- ❑ Code provides instruction to the CPU while data provides the information to be processed.
- ❑ Buses are used to access the code ROM & data RAM memory spaces.
- ❑ Early computer used the same buses for both code & data - **Von Neumann**
- ❑ Such architecture slow down the processing speed of the CPU....

Harvard Architecture in Modern CPUs

- ❑ Separate buses are used to access code and data- Harvard Architecture
- ❑ Modern CPUs are modified Harvard: split L1 instruction and L1 data caches feed a unified L2/L3 and a single main memory.
- ❑ So the programmer sees one Von Neumann memory, while the hardware fetches code and data in parallel.

Modified Harvard: How Real CPUs Split Code and Data



Where This Takes You Next

You now know: number systems, buses, registers, memory organization and the fetch–execute cycle.

Assembly

Write directly to the registers you just studied.

Operating Systems

Memory maps, interrupts and address spaces.

Architecture

Pipelining, cache design and performance.

Parallel & AI

GPUs, SIMD and memory-bound ML workloads.

“Master the machine underneath, and every language, framework and model you meet afterwards becomes a detail rather than a mystery.”